Containers Fundamentals



DRIM

Why Containers?

Containers

Containers are a technology that provides a lightweight and portable way to package and deploy software applications and their dependencies. They offer several benefits and are widely used in modern software development and deployment.

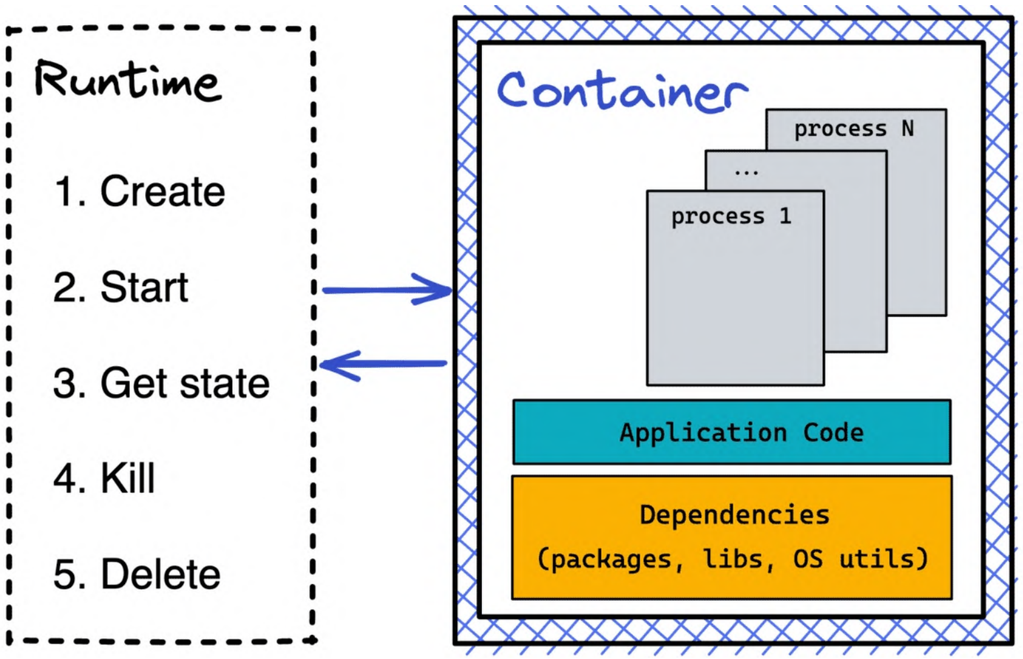
Containers

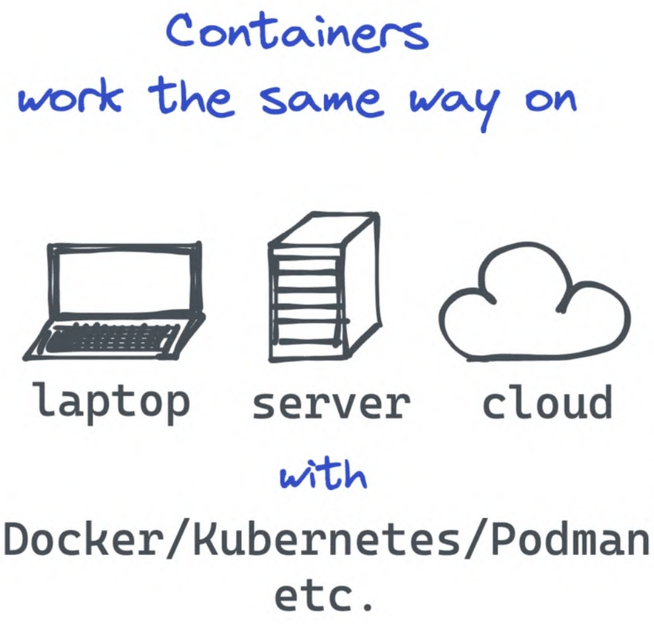
- **Isolation**: Containers provide a high level of application isolation, allowing you to run multiple applications or services independently on the same host.
- **Portability**: Containers encapsulate the application and all its dependencies into a single package.
- **Scalability**: Containers enable horizontal scalability, allowing you to quickly scale up or down your application based on demand.
- **Efficiency**: Containers are lightweight and start up quickly, enabling efficient resource utilization.

Containers

- **Dependency management:** Containers allow you to package your application with its specific dependencies, ensuring that it runs consistently regardless of the underlying host environment.
- **DevOps and CI/CD:** Containers are well-suited for modern software development practices such as DevOps and continuous integration/continuous deployment (CI/CD). Containers provide a consistent environment throughout the development lifecycle, making it easier to build, test, and deploy applications in an automated and reproducible manner.
- **Collaboration and sharing**: Containers provide a standardized format for packaging and distributing applications.

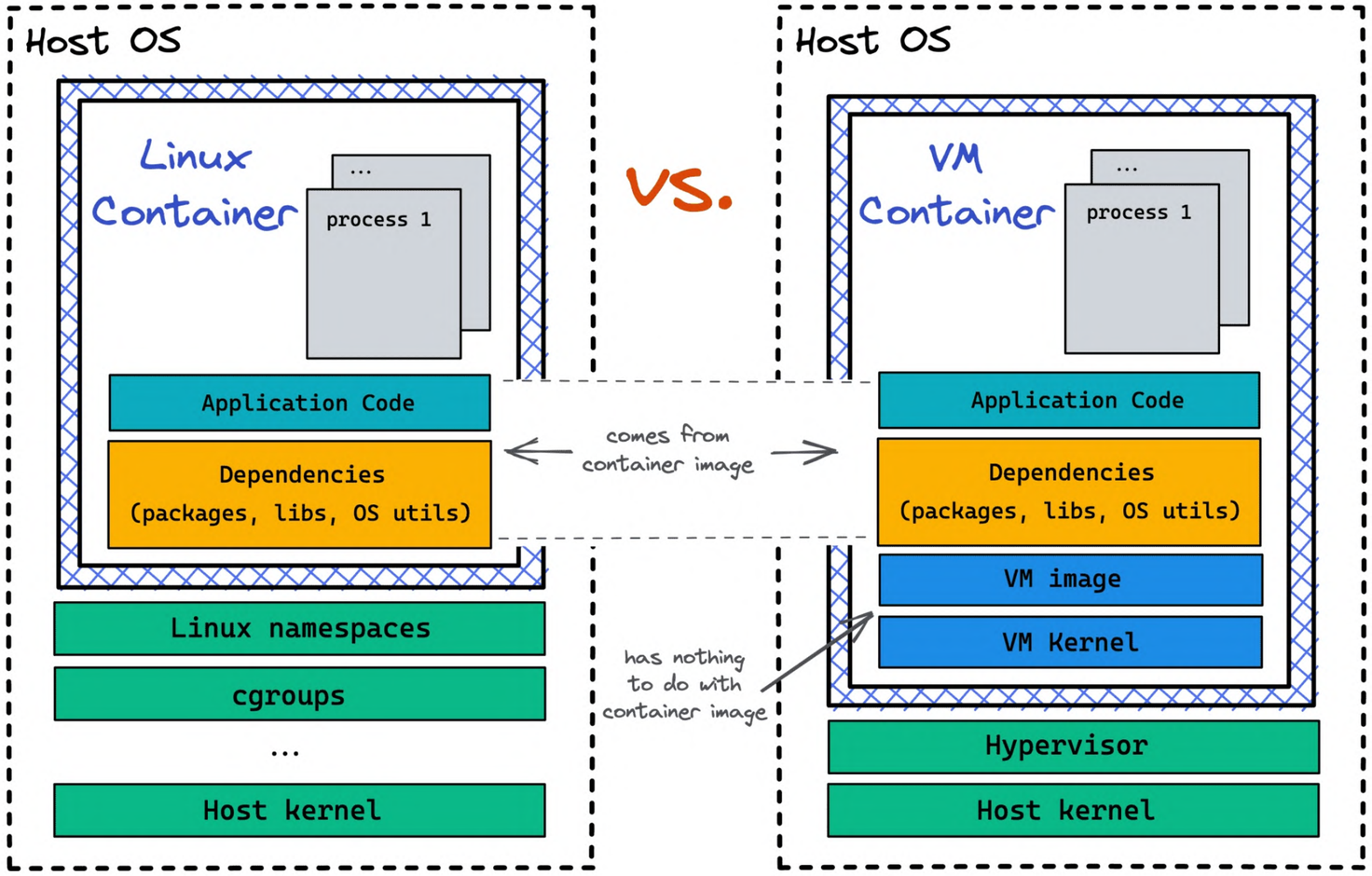
Containers





Drawing by Ivan Velichko

Containers vs VMs



Drawing by Ivan Velichko

DRIM

Linux Namespaces

Namespaces

Linux namespaces are a feature of the Linux kernel that allows for the creation of isolated and independent environments, known as namespaces. Each namespace provides a unique instance of a specific system resource, such as process IDs, network interfaces, mount points, user IDs, and more. These namespaces ensure that processes running within them have their own isolated view of the system, preventing interference and providing enhanced security and resource management.

Namespaces

- **PID namespace**: isolation of the system process tree
- **NET namespace**: isolation of the host network stack
- **MNT namespace**: isolation of host filesystem mount points
- **UTS namespace**: isolation of hostname
- **IPC namespace**: isolation for interprocess communication utilities (shared segments, semaphores)
- **USER namespace**: isolation of system users IDs
- **CGROUP namespace**: isolation of the virtual cgroup filesystem of the host

runc

runc

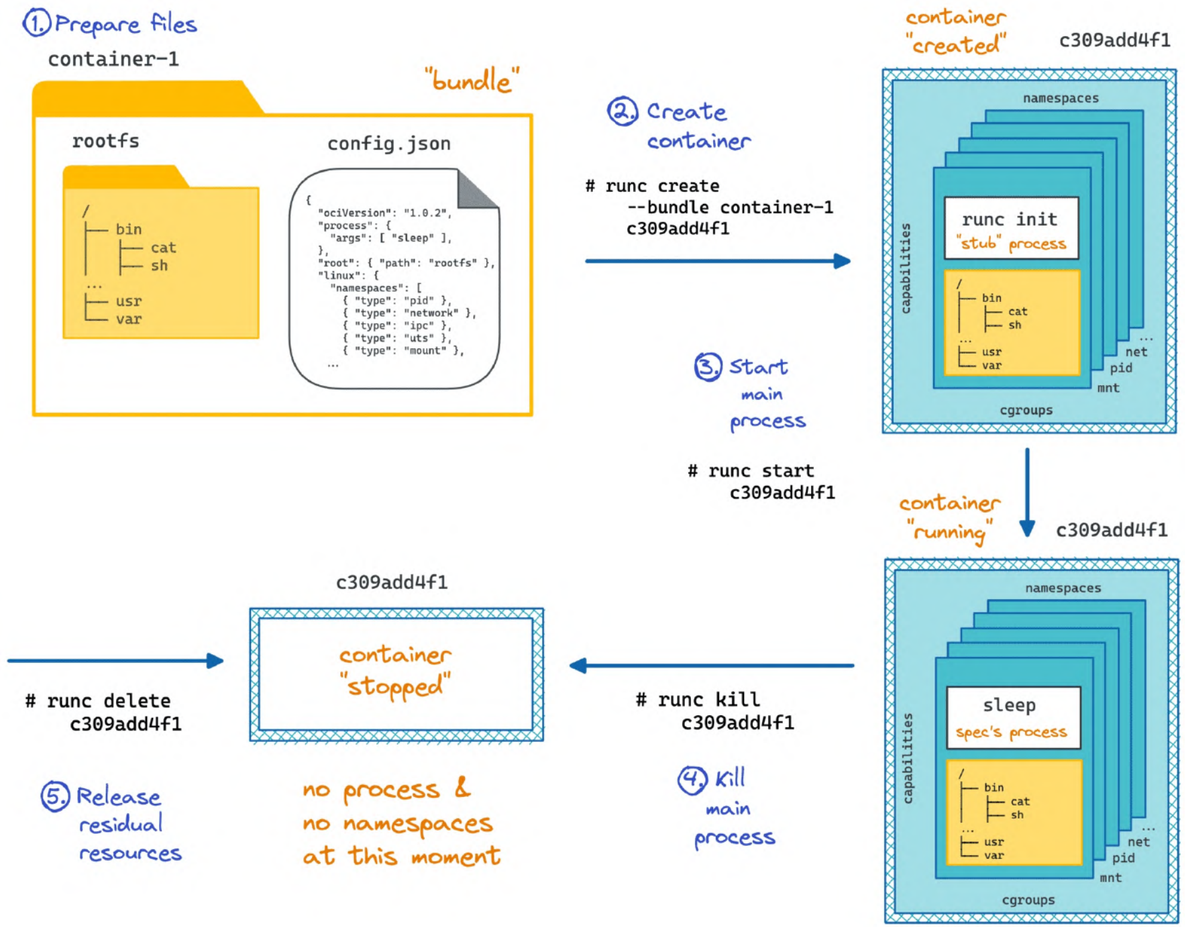
runc is a CLI tool for spawning and running containers on Linux according to the OCI specification.

With runc, you can create, start, stop, and delete containers, as well as manage their namespaces, control groups, and file systems. It allows you to specify various container settings, such as process isolation, resource constraints, and network configurations.

https://github.com/opencontainers/runc

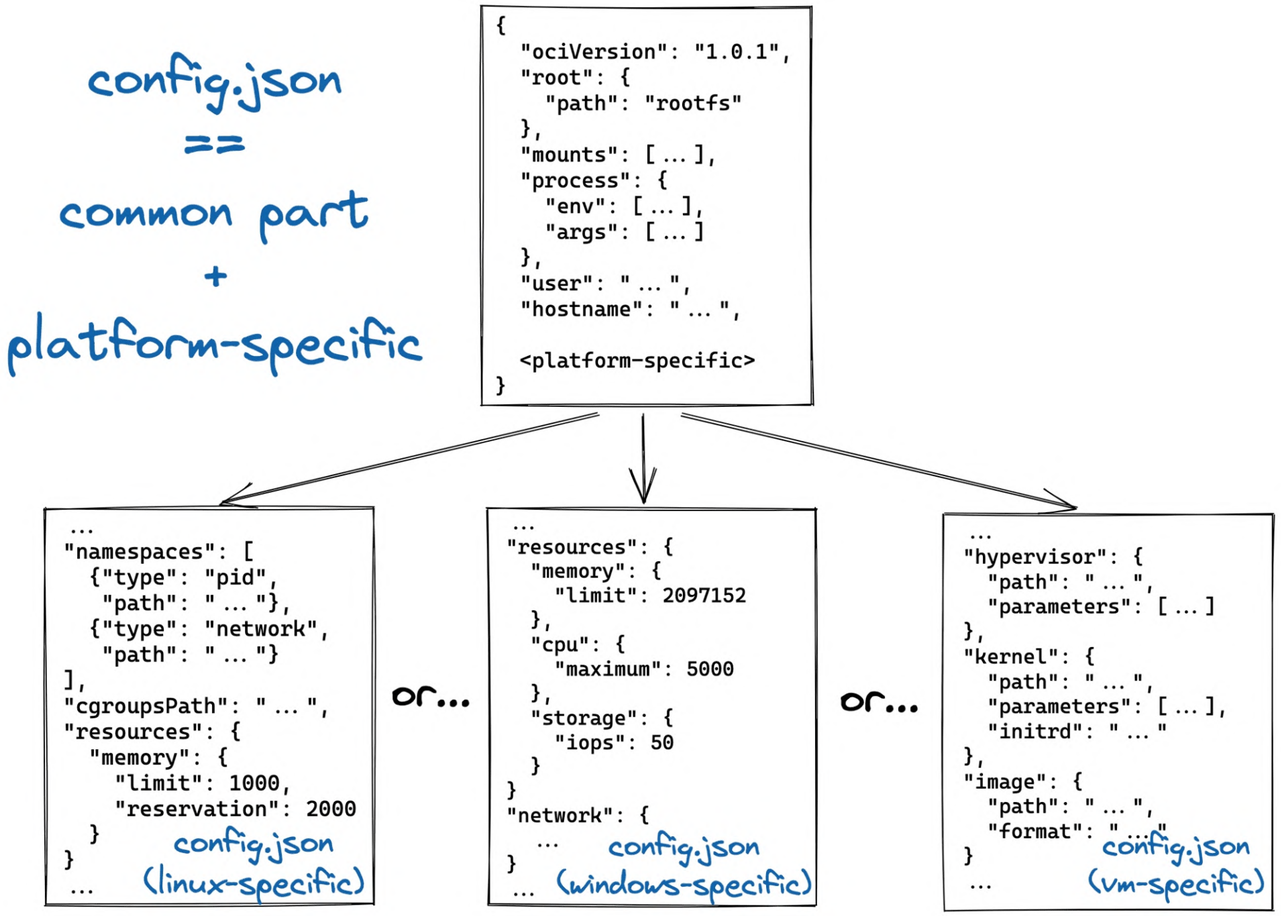
DRIM

runc bundle



Drawing by Ivan Velichko

config.json



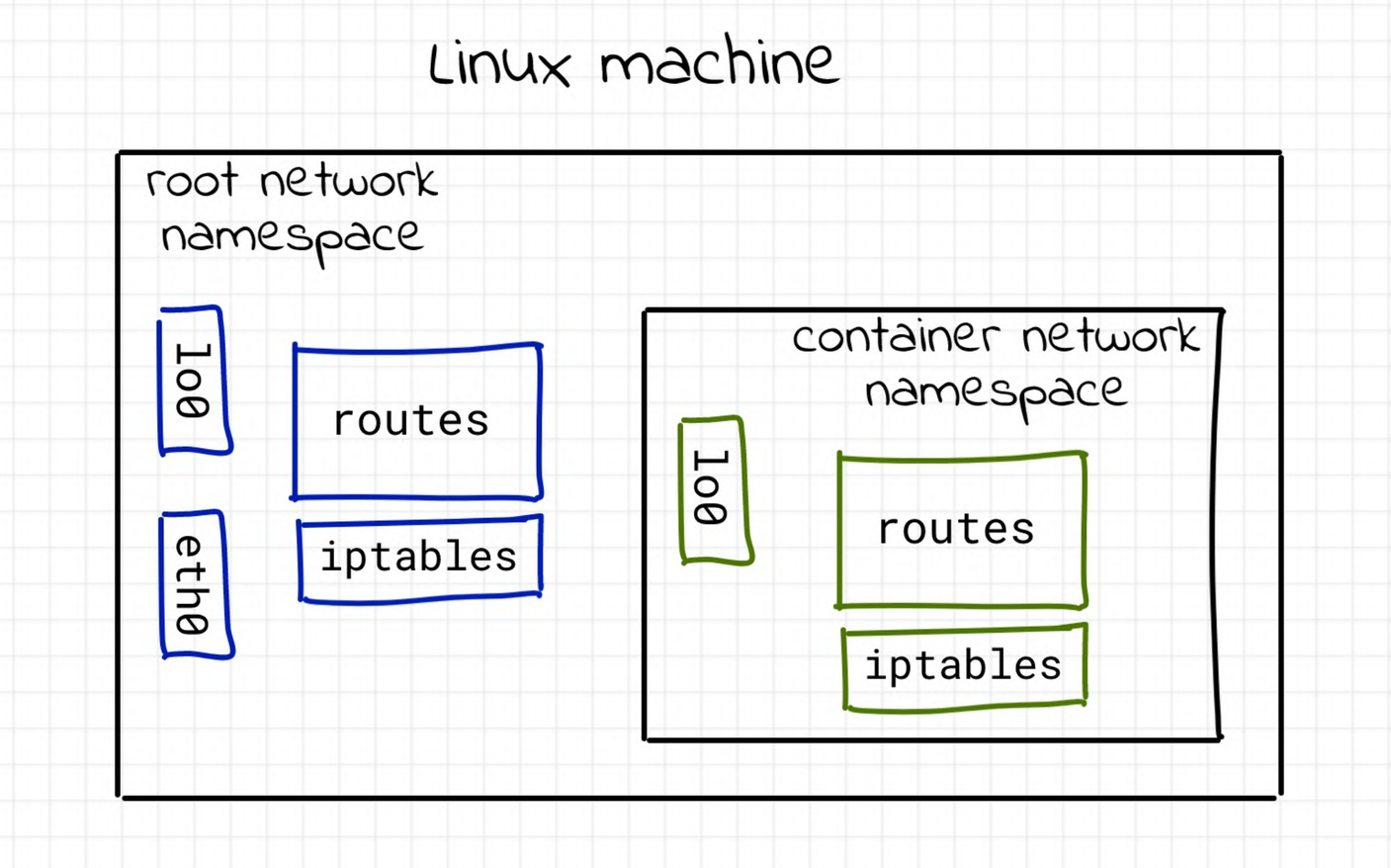
Drawing by Ivan Velichko

DRIM

Networking

DRIM

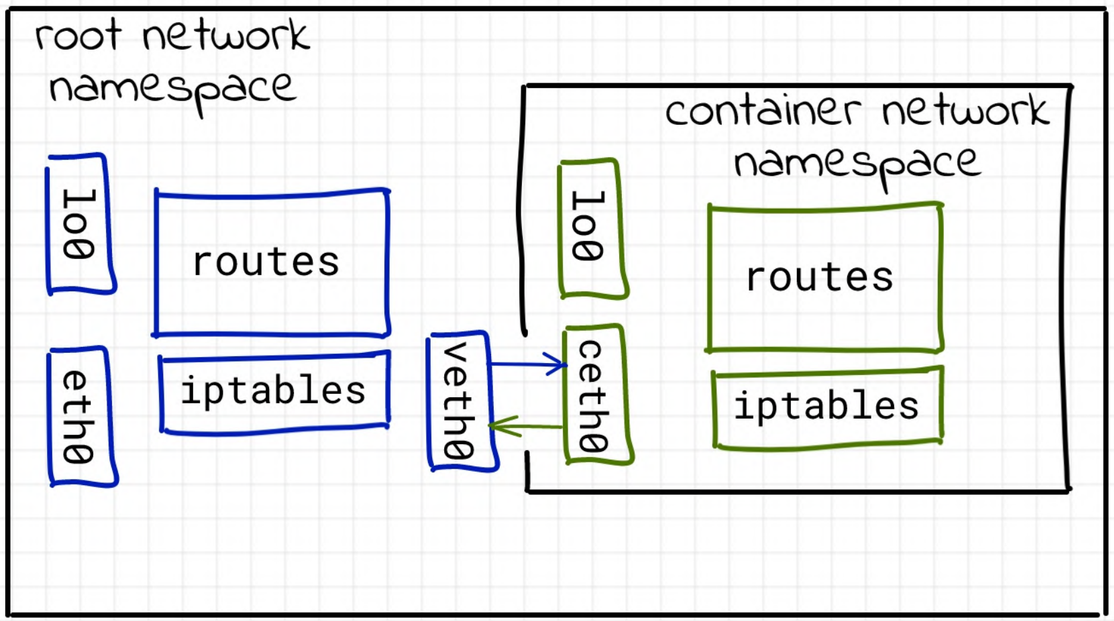
Network Namespaces



Drawing by Ivan Velichko

DRIM

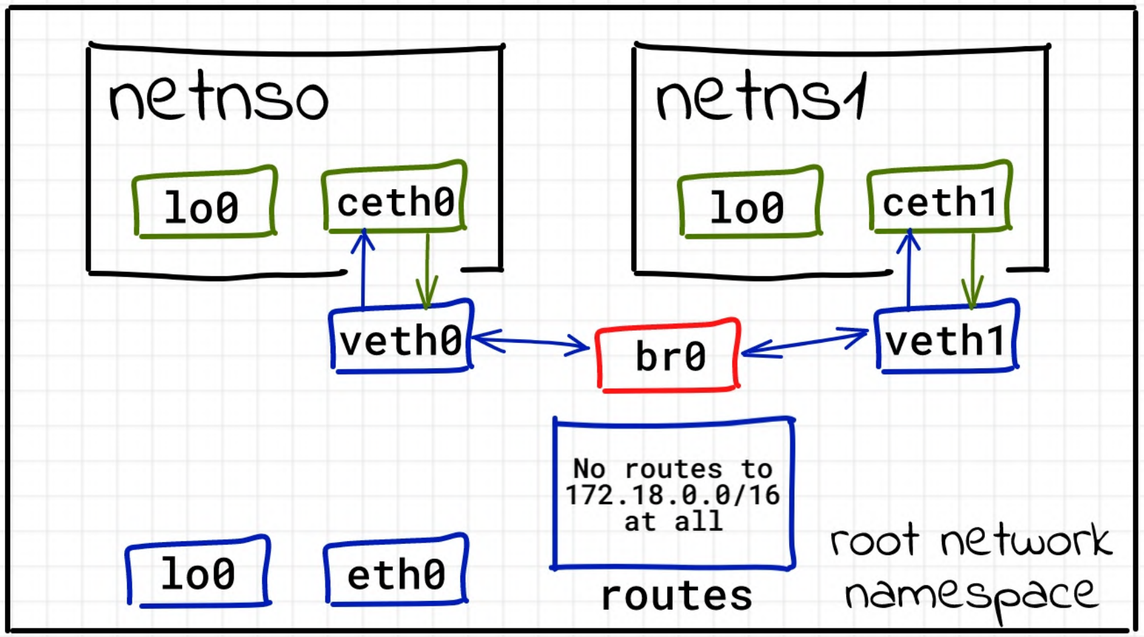
Virtual Ethernet Devices



Drawing by Ivan Velichko

DRIM

Virtual Ethernet Bridge



Drawing by Ivan Velichko

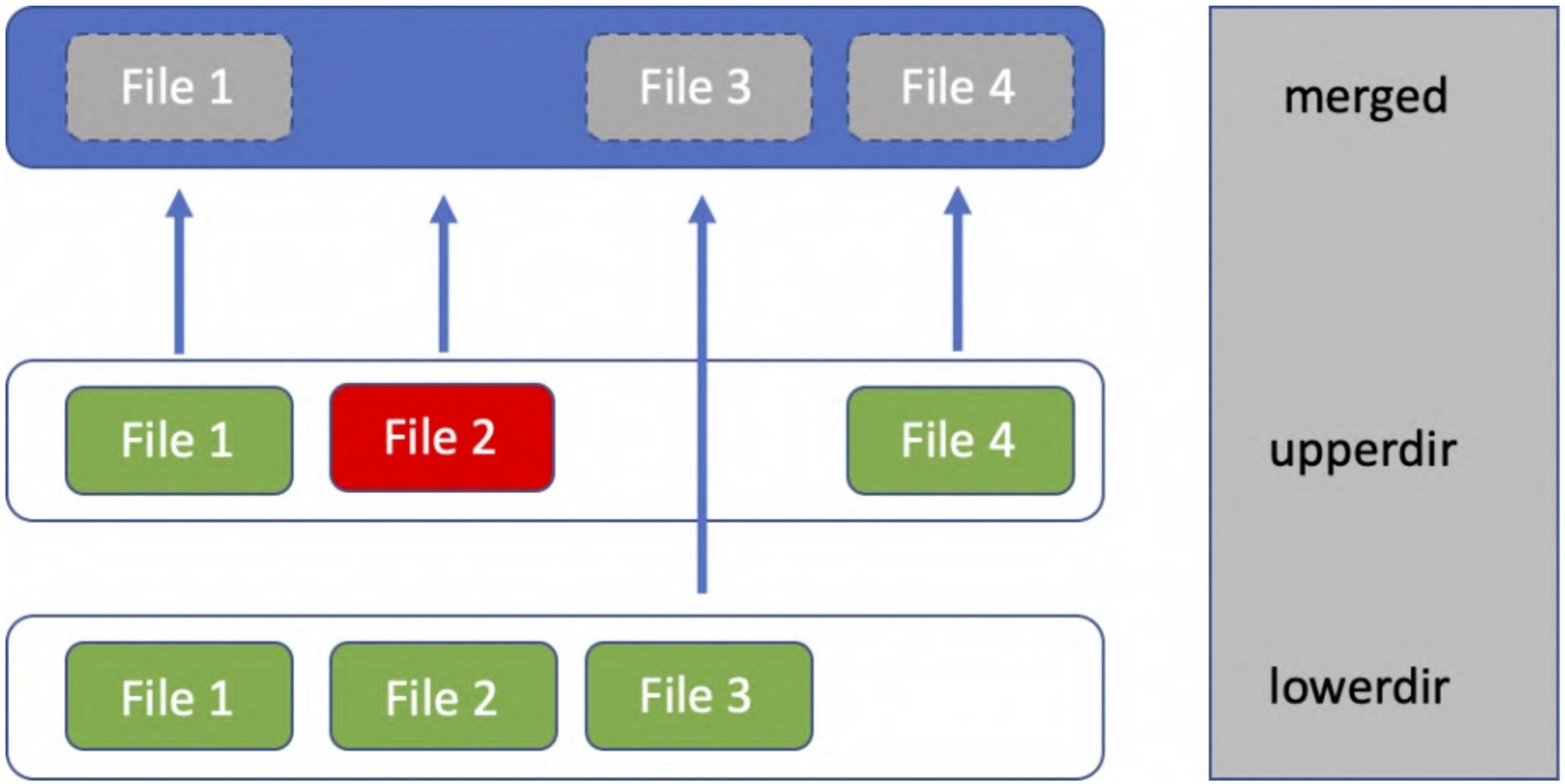
DRIM

Union Filesystems

Union Filesystem

A union filesystem, also known as a union mount or unionfs, is a type of filesystem that allows multiple directories or filesystems to be mounted and presented as a single unified directory structure. It provides a way to overlay multiple directories, making their contents appear as if they were merged together into one directory.

OverlayFS


File 1
File 3
File 4
File 1
File 2
File 4
File 1
File 2
File 3
merged
upperdir
lowerdir

DRIM

Container Images

Container Images

Container images are lightweight, standalone, and executable software packages that include everything needed to run an application, including the code, runtime, system tools, libraries, and dependencies. They provide a consistent and reproducible environment for running applications across different systems.

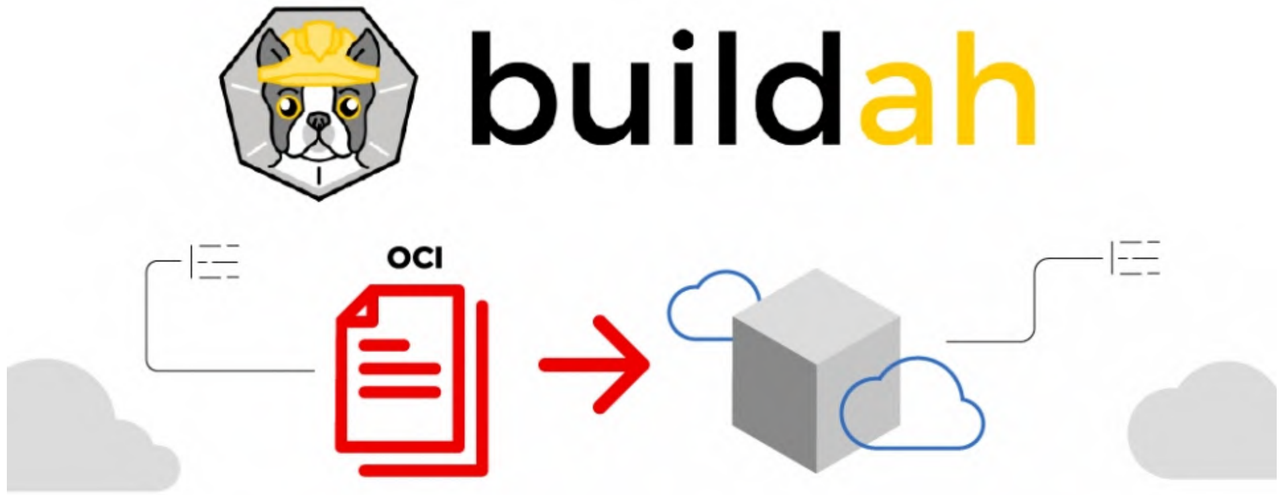
Container Images

- **Portability:** Container images encapsulate the application and its dependencies, making it easy to run the same image on different systems without worrying about compatibility issues.
- **Consistency**: With container images, the application runs in the same environment across different stages of the software development lifecycle, from development to testing and production.
- **Scalability:** Container images can be easily replicated and deployed across multiple instances, allowing for scalable application deployments.

Container Images

- **Isolation:** Containers provide process-level isolation, ensuring that applications running in different containers do not interfere with each other. This isolation enhances security and stability.
- **Versioning and Rollbacks:** Container images can be versioned, allowing for easy rollback to a previous version if issues are encountered. This feature facilitates rapid iteration and deployment of applications.

Buildah



https://github.com/containers/buildah

Buildah

The Buildah package provides a command line tool that can be used to

- create a working container, either from scratch or using an image as a starting point
- create an image, either from a working container or via the instructions in a Dockerfile
- images can be built in either the OCI image format or the traditional upstream docker image format
- mount a working container's root filesystem for manipulation
- unmount a working container's root filesystem
- use the updated contents of a container's root filesystem as a filesystem layer to create a new image
- delete a working container or an image
- rename a local container

DRIM

Thank you! Questions?

